Legal Contract Analysis: NLP Processing Pipeline

Technical Report

HCMUT — Semester 252 (2025 – 2026)

April 30, 2026

Abstract

This report documents a multi-stage NLP pipeline for analysing English legal contracts. The system proceeds through three assignments. Assignment 1 implements a syntactic processing layer comprising clause splitting (spaCy sentence segmentation augmented with rule-based coordinating-conjunction and subordinating-conjunction detection), noun-phrase (NP) chunking with IOB tagging, and full Universal Dependencies parsing. Assignment 2 adds a semantic extraction layer: a custom Named Entity Recognition (NER) model fine-tuned on `nlpaueb/legal-bert-base-uncased` with a rule-based fallback, a dependency-tree-based Semantic Role Labelling (SRL) module, and a dual-model intent classifier (TF-IDF + Logistic Regression and fine-tuned BERT). Assignment 3 integrates all upstream outputs into an end-to-end Retrieval-Augmented Generation (RAG) chatbot backed by ChromaDB, the `all-MiniLM-L6-v2` embedding model, Google Gemini, and a Streamlit web interface.

Contents

Chapter 1

Assignment 1: Syntactic Analysis Pipeline

Legal contracts contain long, complex sentences with nested clauses, coordinating structures, and specialised vocabulary. The first stage of the pipeline decomposes raw contract text into atomic, analysable units and extracts their syntactic structure. Three components are described below: clause splitting, noun-phrase (NP) chunking, and full dependency parsing.

1.1 Clause Splitting

1.1.1 Motivation

A single legal sentence can span several lines and encode multiple independent obligations or conditions. Downstream semantic models (NER, SRL, intent classification) perform significantly better when each input is a single, coherent proposition. The clause splitter reduces complex sentences to their atomic units while preserving the integrity of conditional constructions.

1.1.2 Algorithm

The splitter operates in five sequential stages:

1. **Paragraph segmentation.** The raw input text is partitioned on double newlines, treating each paragraph as an independent processing unit to avoid cross-paragraph sentence boundary errors.

2. **Sentence segmentation.** Each paragraph is processed with spaCy's `en_core_web_sm` model, which detects sentence boundaries more reliably than purely rule-based tokenisers.

3. **Conditional detection.** For each sentence, the algorithm checks whether any token simultaneously satisfies *all three* of the following: (a) its part-of-speech tag is `SCONJ`, (b) its dependency relation is `mark`, and (c) its lower-case form appears in the conditional keyword set (Table 1.1). When this condition is met, the entire sentence is retained as a single indivisible clause. Splitting a conditional sentence would sever the logical dependency between the protasis and apodosis.

4. **Coordinating-conjunction split.** For sentences that do not trigger the conditional guard, the algorithm scans for `CC` tokens whose lower-case form belongs to the coordinate keyword set (Table 1.1). A split is *only* performed when **both** resulting fragments independently contain a subject–verb pair, verified by requiring at least one `nsubj` or `nsubjpass` dependent and one `ROOT` or main-verb token on each side.

5. **Fragment pruning.** Any resulting clause with two tokens or fewer is discarded as a non-informative syntactic fragment.

Table 1.1: Keyword sets governing clause-splitting behaviour.

Set	**Keywords**
Conditional (`SCONJ`)	*if, unless, provided, whereas, subject, upon, when, whenever, although, though, because, since, after, before, until*
Coordinate (`CC`)	*and, but, or, nor, yet, so*

1.1.3 Input/Output Examples

Example 1 — Coordinating split. Input:

> *"The Employer shall pay the Employee a monthly salary of $5,000 and the Employee shall complete all assigned tasks by the deadline."*

Output — two independent clauses, one per line:

1. *The Employer shall pay the Employee a monthly salary of $5,000*
2. *the Employee shall complete all assigned tasks by the deadline*

The conjunction *and* carries the `CC` tag. Both resulting fragments contain independent subject–verb pairs (*Employer/shall pay* and *Employee/shall complete*), so a split is performed.

Example 2 — Conditional retained as single clause. Input:

> *"If the Employee fails to report to work within 3 days without notice, the Employer may terminate this Agreement."*

Output — single clause (no split):

1. *If the Employee fails to report to work within 3 days without notice, the Employer may terminate this Agreement.*

The token *If* carries `SCONJ` POS and `mark` dependency relation, and appears in the conditional keyword set; the sentence is therefore preserved as an atomic unit.

1.2 NP Chunking

1.2.1 Method

Noun-phrase chunking uses spaCy's built-in `doc.noun_chunks` iterator, which returns contiguous base NP spans derived from the dependency parse tree. These spans are converted to the IOB (**I**nside–**O**utside–**B**eginning) tagging scheme defined as follows:

- **B-NP**: the first (or only) token of a noun phrase span.
- **I-NP**: any subsequent continuation token within the same span.
- **O**: a token that does not belong to any noun phrase.

Output is written to `output/chunks.txt` as tab-separated *token*\t*IOB-tag* pairs, one pair per line, with a blank line separating successive clauses.

1.2.2 IOB Tagging Example

Table 1.2 shows the IOB annotation produced for the clause *"The contracting parties shall maintain confidentiality."*.

Table 1.2: IOB tags for *"The contracting parties shall maintain confidentiality."*

Token	IOB Tag
The	B-NP
contracting	I-NP
parties	I-NP
shall	O
maintain	O
confidentiality	B-NP
.	O

The determiner *The*, the pre-modifying participle *contracting*, and the head noun *parties* together form the subject NP. The modal auxiliary *shall* and main verb *maintain* are outside any NP and are tagged `O`. The object NP *confidentiality* begins a new span tagged `B-NP`.

1.3 Dependency Analysis

1.3.1 Parser

Full dependency parsing is performed with spaCy's `en_core_web_sm` model, which implements a transition-based arc-eager dependency parser trained on the English OntoNotes 5 corpus and annotated with Universal Dependencies (UD) relation labels.

1.3.2 Output Format

Each clause is serialised to a JSON object containing the following top-level fields:

- `clause_id` — zero-based integer index of the clause in the document.
- `text` — the original clause string before tokenisation.
- `tokens` — an array of token objects, each carrying:
  - `id`: position index within the clause (zero-based).
  - `token`: surface form as it appears in the text.
  - `lemma`: morphologically normalised form.
  - `pos`: universal POS tag (e.g. `NOUN`, `VERB`).
  - `tag`: fine-grained Penn Treebank tag (e.g. `NN`, `VB`, `MD`).
  - `dep`: Universal Dependencies relation label (e.g. `nsubj`, `dobj`, `ROOT`).
  - `head_id`: position index of the syntactic head token.
  - `head_token`: surface form of the head.

The complete output is written to `output/dependencies.json` as a JSON array of these clause objects.

1.3.3 JSON Output Example

```
{
  "clause_id": 0,
  "text": "The Employer shall pay the salary.",
  "tokens": [
    {"id": 0, "token": "The",      "lemma": "the",
     "pos": "DET",   "tag": "DT", "dep": "det",
     "head_id": 1,   "head_token": "Employer"},
    {"id": 1, "token": "Employer", "lemma": "employer",
     "pos": "NOUN",  "tag": "NN", "dep": "nsubj",
     "head_id": 3,   "head_token": "pay"},
    {"id": 2, "token": "shall",    "lemma": "shall",
     "pos": "AUX",   "tag": "MD", "dep": "aux",
     "head_id": 3,   "head_token": "pay"},
    {"id": 3, "token": "pay",      "lemma": "pay",
     "pos": "VERB",  "tag": "VB", "dep": "ROOT",
     "head_id": 3,   "head_token": "pay"},
    {"id": 4, "token": "the",      "lemma": "the",
     "pos": "DET",   "tag": "DT", "dep": "det",
     "head_id": 5,   "head_token": "salary"},
    {"id": 5, "token": "salary",   "lemma": "salary",
     "pos": "NOUN",  "tag": "NN", "dep": "dobj",
     "head_id": 3,   "head_token": "pay"},
    {"id": 6, "token": ".",        "lemma": ".",
     "pos": "PUNCT", "tag": ".",  "dep": "punct",
     "head_id": 3,   "head_token": "pay"}
```

```
  ]
}
```

Listing 1.1: Dependency parse output for the clause *"The Employer shall pay the salary."*

The ROOT of the clause is *pay* (index 3). *Employer* is its nominal subject (`nsubj`), *shall* is the auxiliary verb (`aux`), and *salary* is the direct object (`dobj`). The determiner *the* (index 0) modifies *Employer* via the `det` relation, while *the* (index 4) modifies *salary* similarly.

Chapter 2

Assignment 2: Semantic Extraction

Building on the syntactic pipeline of Assignment 1, this stage extracts *semantic* information from each clause: named entities that identify the parties, values, and legal references involved; semantic roles that describe who does what to whom; and an intent label that captures the functional type of the clause.

2.1 Custom Named Entity Recognition (NER)

2.1.1 Entity Schema

Six domain-specific entity types are defined for legal contract NER:

- **PARTY** — contracting parties and organisations (e.g. *Party A*, *ABC International Co., Ltd.*).
- **MONEY** — monetary amounts with or without currency symbols (e.g. *USD 10,000*, *$5,000*).
- **DATE** — absolute and relative date expressions (e.g. *05 May 2024*, *within 30 days*).
- **RATE** — percentage or per-unit rates (e.g. *1% per month*, *2% per annum*).
- **PENALTY** — penalty clauses and liquidated-damages expressions.
- **LAW** — statutory references, ordinances, and legal codes (e.g. *Civil Code 2015*).

The full BIO label set used during model training is:

`O, B-PARTY, I-PARTY, B-MONEY, I-MONEY, B-DATE, I-DATE, B-RATE, I-RATE, B-PENALTY, I-PENALTY, B-LAW, I-LAW`

2.1.2 Model Architecture

Primary model — Legal-BERT fine-tuned NER. The primary model fine-tunes `nlpaueb/legal-bert-base-uncased` [1] for token classification using the HuggingFace `Trainer` API. Key training hyperparameters are summarised in Table 2.1.

Table 2.1: Legal-BERT NER training configuration.

Hyperparameter	Value
Base model	`nlpaueb/legal-bert-base-uncased`
Epochs	3
Learning rate	2×10^{-5}
Batch size	8
Train/eval split	90 / 10
Evaluation metric	seqeval (entity-level F1)

Fallback — Rule-based NER. When Legal-BERT confidence falls below a configurable threshold, a `RuleBasedNER` module is invoked. It applies hand-crafted regular expressions tailored to each entity type — for example, monetary patterns of the form `(USD|VND|\$)\s*[\d,]+` for MONEY, and ISO-format or natural-language date patterns for DATE.

2.1.3 Output Format

Results are written to `output/ner_results.json` as a list of objects:

```
[
  {
    "clause_id": 0,
    "text": "Party A shall pay Party B USD 10,000 by 05 May
       2024...",
    "entities": [
      {"text": "Party A",     "label": "PARTY",  "start":  0, "
         end":  7},
      {"text": "Party B",     "label": "PARTY",  "start": 16, "
         end": 23},
      {"text": "USD 10,000",  "label": "MONEY",  "start": 24, "
         end": 34},
      {"text": "05 May 2024", "label": "DATE",   "start": 38, "
         end": 49},
      {"text": "1% per month","label": "RATE",   "start": 53, "
         end": 65}
    ]
  }
]
```

Listing 2.1: NER output format (abbreviated).

2.1.4 NER Example

Table 2.2 shows the entities extracted from a representative legal clause.

Table 2.2: Entities extracted from *"Party A shall pay Party B USD 10,000 by 05 May 2024 at 1% per month penalty rate."*

Entity Text	Label	Start	End
Party A	PARTY	0	7
Party B	PARTY	16	23
USD 10,000	MONEY	24	34
05 May 2024	DATE	38	49
1% per month	RATE	53	65

2.2 Semantic Role Labelling (SRL)

2.2.1 Approach

Rather than a dedicated BERT-based SRL model, the system uses a *dependency-tree-based* approach built on top of spaCy's `en_core_web_sm` parser. For each clause, the `ROOT` token (typically the main verb) is identified as the predicate, and semantic roles are assigned to its dependents by mapping Universal Dependencies labels to PropBank-style argument roles, as shown in Table 2.3.

Table 2.3: Mapping from Universal Dependencies labels to semantic roles.

UD Dependency Label	Semantic Role	Notes
`nsubj`	Agent	Active-voice subject
`nsubjpass`	Theme	Passive-voice subject
`dobj` / `obj`	Theme	Direct object
`iobj`	Recipient	Indirect object
`prep` (to-)	Recipient	Prepositional recipient
Temporal prepositions	Time	`prep` with time head
`advcl` + cond. mark	Condition	Subordinate conditional clause

2.2.2 Output Format

Results are written to `output/srl_results.json` as a list of objects with the following structure:

```
[
  {
    "clause_id": 0,
    "text": "The Employer shall pay the Employee the monthly
       salary ...",
    "predicate": "pay",
    "roles": {
      "Agent":     "The Employer",
      "Theme":     "the monthly salary",
      "Recipient": "the Employee",
      "Time":      "within 30 days"
    }
```

```
  }
]
```

Listing 2.2: SRL output format.

2.2.3 SRL Example

Input:

"The Employer shall pay the Employee the monthly salary within 30 days."

Output:

Predicate:	*pay*
Agent:	*The Employer*
Theme:	*the monthly salary*
Recipient:	*the Employee*
Time:	*within 30 days*

The ROOT token *pay* is the predicate. *The Employer* is the active nominal subject (`nsubj`) mapped to Agent; *the monthly salary* is the direct object (`dobj`) mapped to Theme; *the Employee* is reached via the indirect object (`iobj`) relation mapped to Recipient; and *within 30 days* is a temporal prepositional phrase mapped to Time.

2.3 Intent Classification

2.3.1 Task Definition and Label Set

Intent classification assigns each clause one of four functional intent labels:

- **Obligation** — a party is required to perform an action (keywords: *shall, must, is obligated to*).
- **Prohibition** — an action is forbidden (keywords: *shall not, must not, is prohibited*).
- **Right** — a party is entitled to perform an action (keywords: *may, has the right to, is entitled to*).
- **Termination Condition** — the contract terminates upon an event (keywords: *terminates, expires, is terminated upon/if*).

A priority order — Prohibition > Termination Condition > Right > Obligation — is used to resolve clauses matching multiple keyword patterns during weak-label generation.

2.3.2 Model 1 — TF-IDF + Logistic Regression

Training labels are generated by a keyword-regex weak labeller. A TF-IDF vectoriser converts each clause to a feature vector, and a Logistic Regression classifier is trained on these vectors. Table 2.4 summarises the configuration.

Table 2.4: TF-IDF + Logistic Regression configuration.

Component	Setting
TfidfVectorizer	
n-gram range	$(1, 3)$
max features	5 000
sublinear TF	`True`
LogisticRegression	
Regularisation C	1.0
Solver	`lbfgs`
Multi-class	`multinomial`
Max iterations	1 000

2.3.3 Model 2 — Fine-tuned BERT

The second classifier fine-tunes `bert-base-uncased` as a sequence classifier using `AutoModelForSequenceClassification`. Table 2.5 summarises training settings.

Table 2.5: BERT intent classification training configuration.

Hyperparameter	Value
Base model	`bert-base-uncased`
Epochs	3
Learning rate	2×10^{-5}
Max token length	256

2.3.4 Output Format

Results are written to `intent_classification.txt` as a TSV file with columns `clause_text`, `tfidf_label`, `bert_label`, and `confidence`.

2.3.5 Comparative Example

Table 2.6 compares the predictions of both models on representative clauses.

Table 2.6: Intent classification results for sample clauses.

Clause Excerpt	TF-IDF	BERT	Conf.
"... shall pay..."	Obligation	Obligation	0.96
"... shall not disclose..."	Prohibition	Prohibition	0.98
"... may terminate..."	Right	Termination condition	0.72
"... terminates upon breach..."	Termination condition	Termination condition	0.95

The *"may terminate"* clause illustrates a common ambiguity: TF-IDF's n-gram features weight *may* heavily toward the Right class, while BERT's contextual represen-

tation captures the termination semantics of the full phrase and predicts Termination Condition. The lower confidence score (0.72) correctly signals the model's uncertainty on this boundary case.

Chapter 3

Assignment 3: RAG-Based Legal Contract Chatbot

The third stage integrates all upstream outputs — structured clauses, NER entities, SRL roles, and intent labels — into an end-to-end Retrieval-Augmented Generation (RAG) chatbot. Users can ask natural-language questions about a legal contract and receive answers grounded in specific, cited clauses.

3.1 System Overview

The system is composed of three principal modules that form a linear pipeline (Table 3.1): a vector store for dense retrieval, an LCEL RAG pipeline for answer generation, and a Streamlit front-end for user interaction.

Table 3.1: RAG system components and their roles.

Module	**Technology**	**Role**
Embedding model	`all-MiniLM-L6-v2`	Dense clause representation
Vector database	ChromaDB (cosine similarity)	Semantic clause retrieval
LLM	Gemini `gemini-1.5-flash`	Grounded answer generation
Orchestration	LangChain LCEL	Prompt + chain management
User interface	Streamlit	Interactive web application

3.1.1 Vector Store and Indexing

3.1.2 Embedding Model

Clause embeddings are produced by `sentence-transformers/all-MiniLM-L6-v2`, a lightweight 384-dimension bi-encoder optimised for semantic similarity tasks. Its small size makes it suitable for real-time encoding of query strings at inference time.

3.1.3 ChromaDB Configuration

The vector store is implemented using ChromaDB's `PersistentClient`, enabling the index to survive application restarts without re-processing the contract. Cosine similarity is used as the distance metric:

```python
import chromadb
from chromadb.config import Settings
from sentence_transformers import SentenceTransformer

encoder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection(
    name="legal_clauses",
    metadata={"hnsw:space": "cosine"},
)
```

Listing 3.1: `vector_store.py` — ChromaDB collection initialisation.

3.1.4 Metadata Schema

Each document stored in ChromaDB carries the following metadata fields derived from the upstream pipeline outputs:

- `ner_entities` — JSON-serialised list of entity spans (text, label, start, end).
- `srl_predicate` — main verb of the clause.
- `srl_roles` — JSON-serialised role dictionary (Agent, Theme, Recipient, Time, Condition).
- `intent_tfidf` — intent label from the TF-IDF classifier.
- `intent_bert` — intent label from the BERT classifier.

Storing this metadata alongside the clause text enables the front-end to present NER and SRL information for each retrieved source without additional inference.

3.2 RAG Pipeline

3.2.1 LangChain LCEL Chain

The retrieval and generation steps are orchestrated via LangChain's composable LCEL (LangChain Expression Language) syntax:

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash",
    temperature=0.2)

chain = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    ("human",  "{question}"),
]) | llm | StrOutputParser()
```

Listing 3.2: `rag_pipeline.py` — LCEL chain definition.

3.2.2 Anti-Hallucination System Prompt

A carefully designed system prompt constrains the model to stay within the retrieved context. The key directives are:

1. **Citation requirement.** Every factual claim must be supported by an explicit clause number (e.g. *"As stated in Clause 7, ..."*).
2. **Refusal policy.** If the answer to the user's question cannot be found in the retrieved clauses, the model must explicitly decline to answer rather than confabulate.
3. **Scope restriction.** The model must answer only from the provided contract context and must not import external legal knowledge as a primary source.

```
SYSTEM_PROMPT = """You are a legal contract assistant.
Answer questions ONLY based on the contract clauses provided
    below.

Rules:
- Cite clause numbers for every claim (e.g. "Clause 3 states
    that...").
- If the answer is NOT found in the provided clauses, respond:
   "I cannot find information about this in the contract."
- Do NOT use external legal knowledge as a primary answer source
    .

Contract clauses:
{context}
"""
```

Listing 3.3: System prompt enforcing citation and refusal.

3.2.3 Multi-Turn Conversation

The `chat()` method maintains a list of `HumanMessage` and `AIMessage` objects to support multi-turn dialogue. The conversation history is prepended to each new request, allowing the model to resolve coreferences across turns (e.g. *"Who does they refer to in the previous answer?"*).

3.2.4 Top-k Retrieval

The number of retrieved clauses is configurable via the `top_k` parameter (default: 5). At query time, the user's question is embedded with the same `all-MiniLM-L6-v2` encoder, and the k clauses with the highest cosine similarity scores are returned from ChromaDB and injected into the system prompt as the context window.

3.3 Streamlit User Interface

3.3.1 Interface Layout

The web application (`app.py`) is built with Streamlit and organised into two panels:

- **Sidebar.**
  - Top-k slider (range 1–10) to control retrieval breadth.
  - *Build Index* button: triggers contract parsing and ChromaDB indexing.
  - *Clear Index* button: deletes the persisted collection to allow re-indexing with a different document.
- **Main panel.**
  - Chat input box for natural-language questions.
  - Conversation history rendered as alternating user/assistant message bubbles.
  - Colour-coded intent badges next to each retrieved source clause (Obligation = blue, Prohibition = red, Right = green, Termination Condition = orange).
  - Expandable *Sources* panel for each assistant response, revealing the retrieved clause text along with its NER entities and SRL roles extracted in the upstream pipeline.

3.3.2 Index Building Workflow

When the user clicks *Build Index*, the application executes the following steps:

1. Load the contract text from the configured file path.
2. Run the full upstream pipeline (clause splitting, NER, SRL, intent classification).
3. Embed all clause texts with `all-MiniLM-L6-v2`.
4. Upsert embeddings, texts, and metadata into the ChromaDB collection.
5. Display a success notification with the total number of indexed clauses.

Once the index is built, query latency is dominated by embedding the question ($\approx$10 ms) and the Gemini API round-trip, making the chatbot responsive for interactive use.

Chapter 4

Discussion and Conclusion

4.1 Summary of Achievements

The three assignments collectively deliver a complete, end-to-end NLP pipeline for legal contract understanding.

Assignment 1 — Syntactic Pipeline. A full syntactic preprocessing layer was built using spaCy's `en_core_web_sm` model. The clause splitter correctly handles conditional sentences (preserving logical dependencies) and coordinates splits (requiring both fragments to be grammatically complete). IOB-tagged NP chunks and full Universal Dependencies parse trees are produced for every clause, providing the structural foundation for all downstream components.

Assignment 2 — Semantic Extraction. A three-layer semantic extraction stack was implemented. The custom NER model combines Legal-BERT fine-tuning with a rule-based fallback to identify six domain-specific entity types (PARTY, MONEY, DATE, RATE, PENALTY, LAW). The dependency-tree-based SRL module extracts five semantic roles (Agent, Theme, Recipient, Time, Condition) without requiring a computationally expensive dedicated SRL model. The dual-model intent classifier provides both an interpretable TF-IDF baseline and a higher-accuracy BERT model across four intent classes.

Assignment 3 — RAG Chatbot. An interactive Streamlit application integrates all upstream outputs. ChromaDB with cosine similarity provides fast, semantically grounded clause retrieval. The LangChain LCEL chain orchestrates embedding, retrieval, and generation via Google Gemini (`gemini-1.5-flash`), with an anti-hallucination system prompt that mandates clause citation and explicit refusal when information is absent.

4.2 Error Analysis

Despite solid overall performance, several systematic error patterns were observed during qualitative evaluation.

Clause Splitting

Long parenthetical enumerations (e.g. *"Responsibilities include: (a) filing reports, (b) attending meetings, (c) maintaining records"*) are sometimes split at the enumeration markers rather than treated as a single obligation list. Additionally, nested coordinating structures where an inner `CC` lacks a clear local subject can produce over-splitting, creating non-sentential fragments that should have been pruned but pass the two-token length guard.

NP Chunking

Prepositional attachment ambiguity affects chunks for expressions such as *"payment of $5,000 per month"*; depending on spaCy's parse, the PP *of $5,000 per month* may be partially incorporated into the NP span, inflating the chunk boundary. Appositive NPs (e.g. *"the Employer, ABC Corp."*) are treated as two separate chunks rather than one co-referential span.

Dependency Parsing

Passive-voice legal constructions (*"shall be deemed to constitute a material breach"*) and modal stacks (*"shall have the right to terminate"*) sometimes produce unusual ROOT assignments. In the latter example, spaCy may select *have* rather than *terminate* as ROOT, misrepresenting the logical predicate.

Named Entity Recognition

The rule-based fallback misses multi-word party names that do not match known patterns (e.g. *"ABC International Co., Ltd."* with trailing legal suffixes). Legal-BERT occasionally confuses RATE and MONEY labels for monetary amounts that appear without explicit percentage signs (e.g. *"at a rate of three dollars per day"*).

Semantic Role Labelling

The dependency-tree approach extracts roles only from the ROOT predicate, so sentences with multiple verbs (e.g. *"The Employer shall have the obligation to pay and report"*) yield roles for only one predicate. Stacked modal constructions (*"shall have the obligation to pay"*) cause the embedded verb *pay* to be missed as a secondary predicate entirely.

Intent Classification

The word *shall* in rights clauses (*"Party A shall have the right to inspect the premises"*) is misclassified as Obligation by both models because *shall* is the dominant Obligation indicator in the training data. BERT exhibits higher confidence on these errors, which is more dangerous than the correctly low-confidence predictions from TF-IDF on the same clauses.

4.3 Future Work

Clause Splitting

Training a neural sentence-pair model (e.g. fine-tuned BERT) to classify whether two consecutive clauses should be merged would address the enumeration-splitting problem. Enumerated list items (items starting with *(a)*, *(b)*, *(i)*, etc.) should be detected by a pre-processing rule and treated as atomic units before dependency-based splitting is applied.

Named Entity Recognition

Annotating a larger domain-specific dataset with the six-entity schema would substantially improve generalisation. Exploring publicly available legal NER benchmarks — such as LexNER [3] or CUAD [2] — could provide additional fine-tuning signal. Incorporation of a gazetteer for known company-name suffixes (*Ltd., Corp., S.A., GmbH*) would improve recall for multi-token PARTY entities.

Semantic Role Labelling

Integrating a proper BERT-based SRL model (e.g. AllenNLP's structured-prediction SRL or a PropBank-fine-tuned sequence labeller) would handle multi-predicate sentences and complex nested argument structures that the current dependency-tree approach cannot resolve.

Intent Classification

Adding a fifth class — **Definition** — for clauses that define terms (e.g. *"'Business Day' means any day other than a Saturday..."*) would reduce label noise in the current four-class setting. Experimenting with legal domain-specific pre-training — such as ContractBERT or LegalBERT variants trained on larger contract corpora — may further improve accuracy, particularly on boundary cases between Right and Obligation.

RAG Chatbot

Several enhancements would improve retrieval quality and answer fidelity: (i) adding a cross-encoder re-ranker [4] on top of the initial top-k retrieval to improve precision; (ii) extending the citation mechanism to include precise character offsets rather than clause indices; (iii) supporting multi-document contracts with document-level metadata filters; and (iv) evaluating the chatbot's faithfulness and relevance using automated RAG evaluation frameworks such as RAGAS [5].

Bibliography

[1] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, *LEGAL-BERT: The Muppets straight out of Law School*, in *Findings of EMNLP 2020*, pp. 2898–2904, 2020.

[2] D. Hendrycks, C. Burns, A. Chen, and S. Ball, *CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review*, *arXiv:2103.06268*, 2021.

[3] P. de Araujo Luz de Araujo, C. Niklaus, and M. Stürmer, *LEXTREME: A Multi-Lingual and Multi-Task Benchmark for the Legal Domain*, *arXiv:2301.13126*, 2021.

[4] R. Nogueira and K. Cho, *Passage Re-ranking with BERT*, *arXiv:1901.04085*, 2019.

[5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, *RAGAS: Automated Evaluation of Retrieval Augmented Generation*, in *EACL 2024*, 2024.